

Semestre: 2026.1

Disciplina:

**ARA0062 DESENV. WEB EM HTML5, CSS,
JAVASCRIPT E PHP**

Aula 11

Fortaleza, CE: 18 de maio, 2026



- 1.Entrada e Saída de Dados
- 2.Condicionais (if/else)
- 3.Loops (for, while)
- 4.Arrays
- 5.Funções ← Vamos desenvolver este
tópico
- 6.Strings
- 7.Projeto Final





O que é uma condição?

No sentido coloquial é uma situação ou um requisito que precisa ser atendido para que algo aconteça.

Se tiver cachaça, eu vou para a festa.

E na programação?

Na **programação**, é uma expressão que pode ser avaliada como **verdadeira** ou **falsa** e que determina o fluxo da execução do código.

Condição simples.

Dessa forma, uma **condicional simples** executa um bloco de código apenas **se** uma condição for verdadeira, utilizando **if** (definida como uma estrutura de controle de fluxo)

CONDICIONAIS - SIMPLES

Em linguagem de programação de computadores, por exemplo (JS) :

É a forma mais básica de estrutura condicional em JavaScript. Ela é usada quando queremos executar um bloco de código apenas se uma determinada condição for verdadeira.

```
if (condition) {  
    // block of code to be executed if the condition is true  
}
```

VEJAMOS O EXEMPLO A SEGUIR:

```
<script>  
    let time = "Fortaleza SC"  
  
    if(time == "Fortaleza SC"){  
        window.alert("Você torce para o Fortaleza!")  
    }  
</script>
```

Desafio 01:

A partir do exemplo anterior, modifique a condição para time “Ceará SC”;

```
<script>  
  let time = "Fortaleza SC"  
  
  if(time == "Fortaleza SC"){  
    window.alert("Você torce para o Fortaleza!")  
  }  
</script>
```

Desafio 02:

Use prompt para interação com usuário:

```
<script>
  // Usando o prompt
  let timeEstimacao = window.prompt("Digite o nome do seu time: ")

  if(timeEstimacao == "Fortaleza SC"){
    window.alert("Você torce para o Fortaleza!")
  }
</script>
```

CONDICIONAIS - COMPOSTAS

É uma estrutura de controle que permite executar diferentes blocos de código com base em múltiplas condições. A sintaxe é:

```
if (condition1) {  
    // block of code to be executed if condition1 is true  
} else if (condition2) {  
    // block of code to be executed if the condition1 is false and condition2 is true  
} else {  
    // block of code to be executed if the condition1 is false and condition2 is false  
}
```

```
<script>
```

```
let time = window.prompt("Digite o nome do seu time:");
```

```
if(time == "Fortaleza SC"){
```

```
    window.alert("Você torce para o Fortaleza!");
```

```
}else{
```

```
    window.alert("Você torce para o Ceará!");
```

```
}
```

```
</script>
```


CONDICIONAIS – COMPOSTAS COM MÚLTIPLAS ALTERNATIVAS

São estruturas de controle que permitem testar várias condições e executar diferentes blocos de código com base no resultado.

A sintaxe é a seguinte:

```
// if (condição1) {  
//   // código a ser executado se condição1 for verdadeira  
// } else if (condição2) {  
//   // código a ser executado se condição2 for verdadeira  
// } else if (condição3) {  
//   // código a ser executado se condição3 for verdadeira  
// } else {  
//   // código a ser executado se nenhuma das condições anteriores for verdadeira  
// }
```

BOLAR UM PLANO PARA SAIR....

```
<script>  
  
  let time = window.prompt("Digite o nome do seu time:")  
  
  if(time == "Fortaleza SC"){  
    window.alert("Você torce para o Fortaleza!")  
  }else if(time == "Ceará SC"){  
    window.alert("Você torce para o Ceará!")  
  }else if(time == "Flamengo"){  
    window.alert("Você torce para o Flamengo!")  
  }else{  
    window.alert("Time desconhecido!")  
  }  
  
</script>
```



LOADING



Loops em JavaScript

São estruturas de repetição utilizadas para executar um bloco de código várias vezes. Evitam repetição manual de comandos.

LOOPS

O loop **for** é utilizado quando sabemos quantas vezes o código deve repetir.

SINTAXE:

```
for(inicialização; condição; incremento){  
  
    // código  
  
}
```

Sem usar for

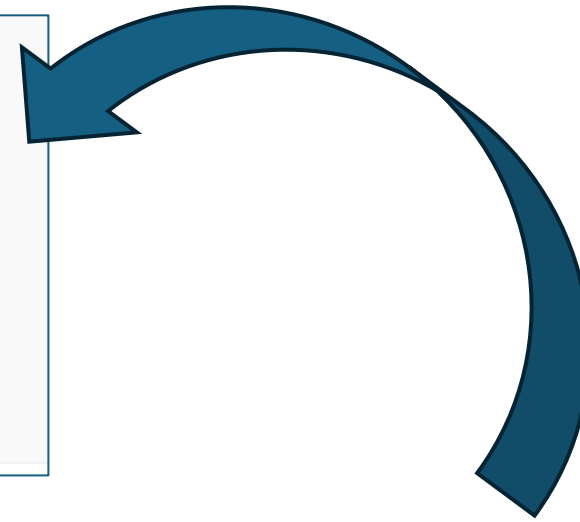
Sem estrutura de repetição, precisaríamos repetir o comando manualmente.

```
<script>  
    // Esta seria a forma de repetir um código 5 vezes sem usar o FOR  
    console.log("Fortaleza")  
    console.log("Fortaleza")  
    console.log("Fortaleza")  
    console.log("Fortaleza")  
</script>
```

LOOPS

Com o **for**

```
for(inicialização; condição; incremento){  
  
    // código  
  
}
```



Sem o **for**

```
<script>  
    // Esta seria a forma de repetir um código 5 vezes sem usar o FOR  
    console.log("Fortaleza")  
    console.log("Fortaleza")  
    console.log("Fortaleza")  
    console.log("Fortaleza")  
</script>
```

Com o **for**

```
<script>  
    // usando o FOR para repetir um código 5 vezes  
    for (let i = 0; i < 5; i++) {  
        console.log("Fortaleza")  
    }  
</script>
```

Loop **while** ou while

Estrutura de controle que permite repetir um bloco de código enquanto uma condição for verdadeira. Ele é útil quando não sabemos exatamente quantas vezes o código precisa ser executado, mas queremos continuar repetindo até que uma condição específica seja atendida.

```
// Sintaxe:  
// while (condição) {  
//     // código a ser executado  
// }
```

Loop **while** ou while

```
// Sintaxe:  
// while (condição) {  
//     // código a ser executado  
// }
```



Loop **while** ou **while**

Laço de repetição que executa um bloco de código enquanto uma condição for verdadeira.

Enquanto o Fortaleza continuar dando moleza, o Ceará vai levando troféus estadual.



Loop **while** ou while

Laço de repetição que executa um bloco de código enquanto uma condição for verdadeira.

Enquanto o Ceará não for para liberta, o Fortaleza vai continuar sendo o time de elite sul-americana.



Loop **while** ou while

```
<script>

let contador = 1

while(contador <= 5){

    window.alert("Fortaleza")

    contador++

}

</script>
```

Desafio com loop **while**

```
// Exemplo: Tabuada do 7 usando while
// console.log("--- Tabuada do 7 ---");
// let n = 1;
// while (n <= 5) {
//   console.log(`7 x ${n} = ${7 * n}`);
//   n++; // Incrementa n para a próxima multiplicação
```

CONTINUAÇÃO

ARRAYS

São estruturas de dados usadas para armazenar múltiplos valores em uma única variável. Eles são como listas ordenadas que podem conter qualquer tipo de dado (números, strings, objetos, funções, outros arrays, etc.).

```
const frutas = ["maçã", "banana", "laranja"];
```

VARIÁVIES vs ARRAYS

A principal diferença é que variáveis guardam UM valor, enquanto arrays guardam MÚLTIPLOS valores organizados em uma lista.

VARIÁVIES

```
let fruta1 = "maçã";  
let fruta2 = "banana";  
let fruta3 = "laranja";  
  
console.log(fruta1); // "maçã"  
console.log(fruta2); // "banana"  
console.log(fruta3); // "laranja"
```

ARRAYS

```
let frutas = ["maçã", "banana", "laranja"];  
  
console.log(frutas[0]); // "maçã"  
console.log(frutas[1]); // "banana"  
console.log(frutas[2]); // "laranja"
```

VARIÁVIES vs ARRAYS

Quando usar cada um?

Use variável quando:

Tiver um único dado para armazenar ou quando o dado é independente e não se relaciona com outros

```
let nome = "João";  
let idade = 25;  
let estaLogado = true  
let preco = 99.90;
```

VARIÁVIES vs ARRAYS

Quando usar cada um?

Use array quando:

Tiver uma lista ou coleção de dados relacionados; precisar organizar múltiplos itens do mesmo tipo; precisar fazer operações em massa (ordenar, filtrar, mapear) ou tiver número de itens pode variar (dinâmico);

```
let nomes = ["João", "Maria", "Pedro", "Ana"];  
let temperaturas = [23.5, 24.0, 22.8, 25.1];  
let produtos = ["camisa", "calça", "tênis", "meia"];
```

ARRAYS

VARIÁVIES vs ARRAYS

Quando usar cada um?

Vantagens do array sobre múltiplas variáveis:

Problema com variáveis separadas:

Com variáveis:

```
let nota1 = 8.5;  
let nota2 = 7.0;  
let nota3 = 9.5;
```

```
// Calcular média fica inviável manualmente  
let media = (nota1 + nota2 + nota3 + ... + nota100) / 100;
```

Solução com array:

```
let notas = [8.5, 7.0, 9.5, 6.0, 9.0, 10, 7.5, 8.0, 6.5, 8.5];
```

OBS:

Com laço for, é possível percorrer todos os elementos e aplicar operação desejada:

```
let soma = 0;  
for (let i = 0; i < notas.length; i++) {  
    soma += notas[i];  
}  
let media = soma / notas.length;
```



Ou usando reduce:

```
let mediaReduce = notas.reduce((soma, nota) => soma + nota, 0) / notas.length;
```

ARRAYS

```
// Acessando elementos (índice começa em 0)  
console.log(frutas[0]); // "maçã"  
console.log("Total:", frutas.length);
```


ARRAYS

EXERCÍCIOS

Acessem o link <https://donthpad.com/MLKP/exerJS> e localizem os exercícios de 1 a 10.

Procedimentos:

Criem um repositório no GitHub com o nome: EXERCICIOS-JS

Para cada exercício (1 a 10):

- 1. Copiem o código HTML fornecido no Dontpad (contém o script JavaScript)*
- 2. Criem um arquivo separado com o nome exercicio_X.html (onde X é o número do exercício)*
- 3. Adicionem o arquivo ao repositório*
- 4. Após copiar todos os 10 exercícios, realizem o commit e o push para o GitHub.*

Ao final, compartilhar o link no grupo.